Our experimental setup:

For our experiment we ran three different implementations (Pthreads, MPI, and OpenMP) on Kansas State University's BEOCAT. We specifically ran all of our implementations using the mole node which has the following specs.

Processors	2 x 10-Core Xeon E5-2630 v4 --> 20 cores
Ram	32 GB
Hard Drive	1 x 1 TB 7,200 RPM SATA
NICs	2 x Intel I350
QDR Infiniband	QLogic QLE7340

Table 1: Specifications of the mole node on BEOCAT

Source: Kansas State University

The experiment was reading ASCII values from a ~1.7GB wiki dump on BEOCAT to find max ASCII values line to line. The full wiki dump was roughly 1,200,000 lines. Each implementation was tested on four distinct input sizes (1,000, 10,000, 100,000, and ~1,200,000 lines). With each one repeated four times to account for any variation in the tests. The following were the parameters that were used:

1. Thread/Rank Count: 1, 2, 4, 8, 16 (32 would not run so it is left out)
2. Memory Allocation: 64MB, 128MB, 512MB, 1GB, 1.5GB, 3GB
3. Nodes (MPI Only): 1, 2, 4
4. Input Sizes: 1,000, 10,000, 100,000, ~1,200,000

Pthreads Analysis

We analyzed our pthreads implementation by using a ~1.7GB wiki dump on BEOCAT. Experiments were conducted by varying the thread counts (1, 2, 4, 8, 16) and memory allocation (512MB, 1GB, 1.5GB, 3GB) across four distinct input sizes (1,000, 10,000, 100,000, and ~1,200,000 lines).

One of the first things we noticed as we ran multiple tests is that any amount of cores on 64MB did not complete, but were instead killed. We believe this has to do with the lack of memory allocation being properly handled as our test was such a large file. Another notable thing is that most of the time, unless memory was above 1.5GB then 1 thread would not complete.

At the other end of the spectrum we had no tests completed on 32 threads. This was because any attempt by BEOCAT would tell us there were not enough cores to be allocated to the task. Therefore no data was collected.

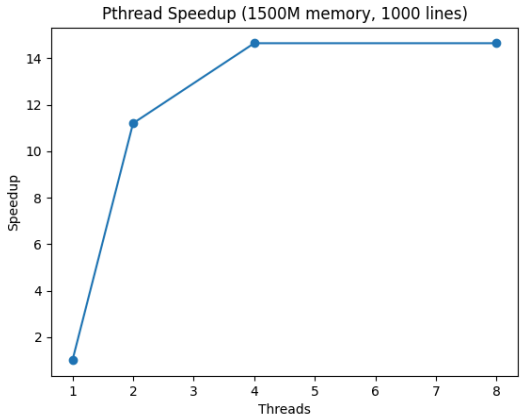
For the smaller input sizes (1,000 and 10,000 lines), runtimes were incredibly fast—often finishing in under 0.01 seconds for 1,000 lines and around 0.03 seconds for 10,000 lines. Because the tasks completed so quickly, increasing the thread count did not yield any meaningful speedup. In fact, the overhead of creating and joining the threads meant that multi-threaded runs often performed identically to the single-threaded baseline.

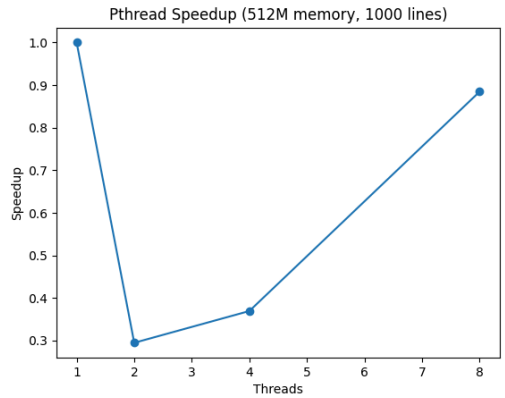
On the medium input size (100,000 lines), we began to see the clear benefits of parallelization. Single-threaded execution hovered around 0.35 to 0.40 seconds, while pushing the thread count to 8 consistently dropped the time down to the 0.22 to 0.25 second range.

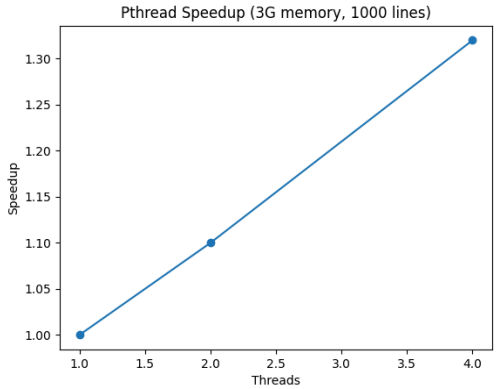
The full input size clearly showed the Pthreads speedup, but also showed environmental volatility on the cluster. Runs with a single thread had a lot of variation, but once threads increased to 4 and 8, the runtimes had much less variation.

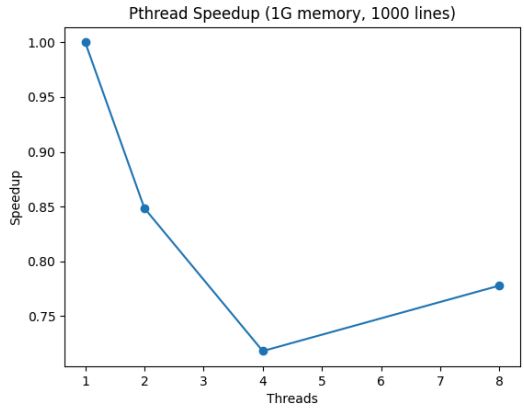
Lastly, unlike MPI, memory had a noticeable impact for Pthreads for the biggest input.

Input Size 1000

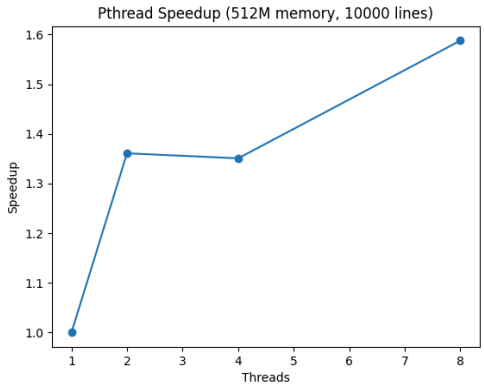


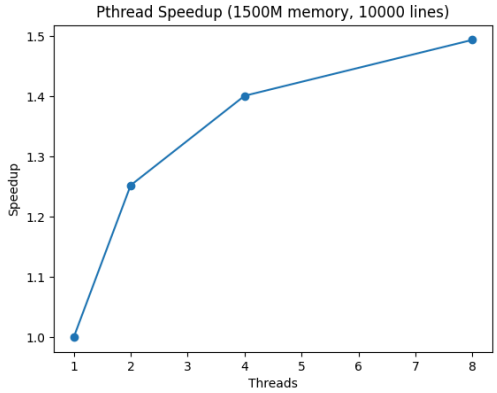


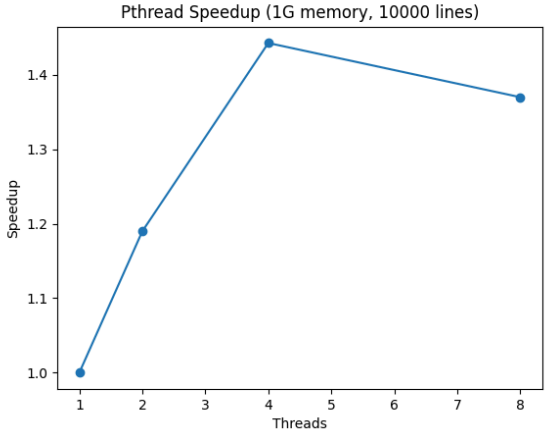


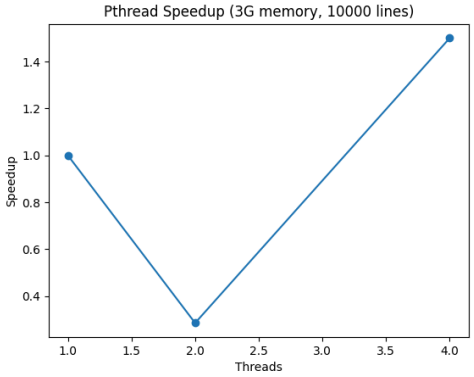


Input Size 10,000

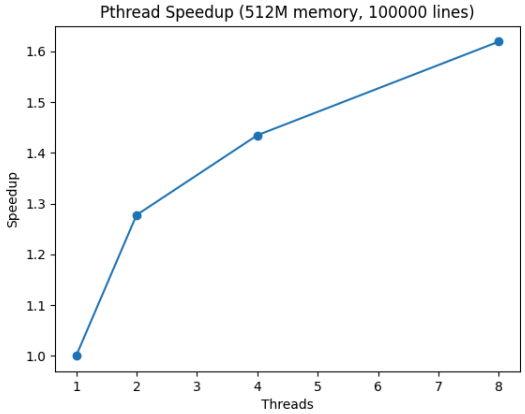


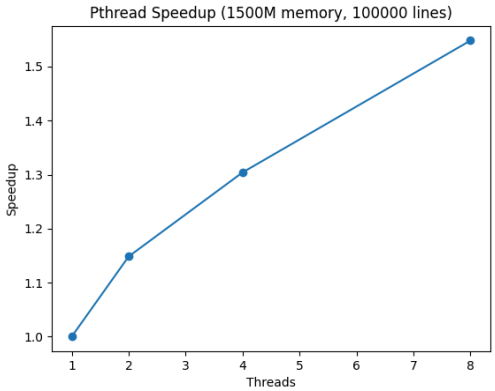


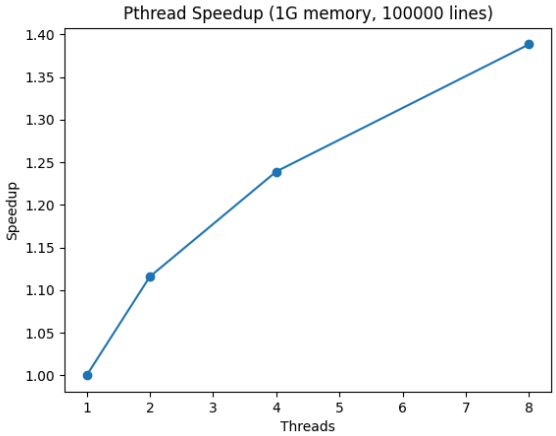


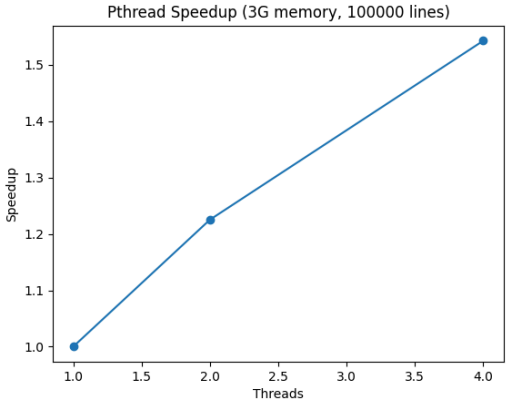


Input Size 100,000

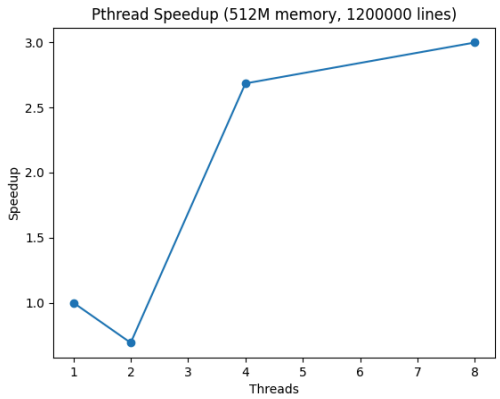


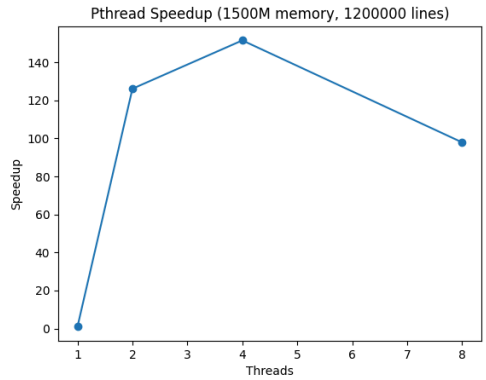


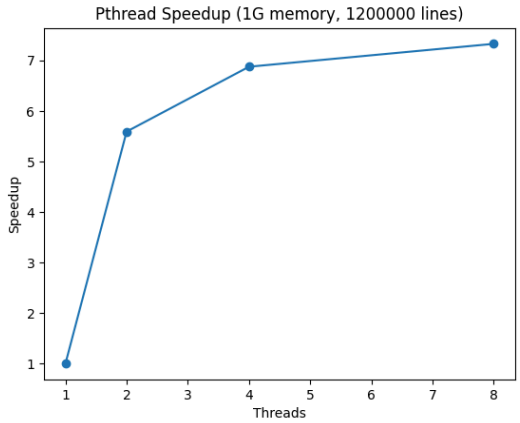


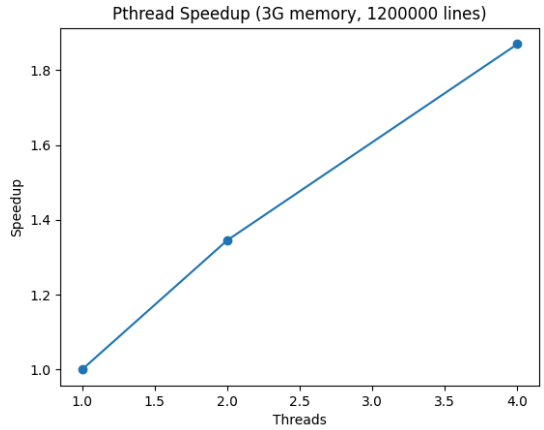


Input Size 1,000,000









MPI Analysis

MPI implementation distributes the workload with the different ranks. This allows the code to break down the task into multiple parts. The different thing about MPI versus pthreads or OpenMP is that each rank has its own private memory to accomplish their tasks.

Similar to our pthreads implementation we could not get any memory with 64MB to complete and were killed. For the 32 ranks we had the same problem with BEOCAT not finding enough resources for us to test it.

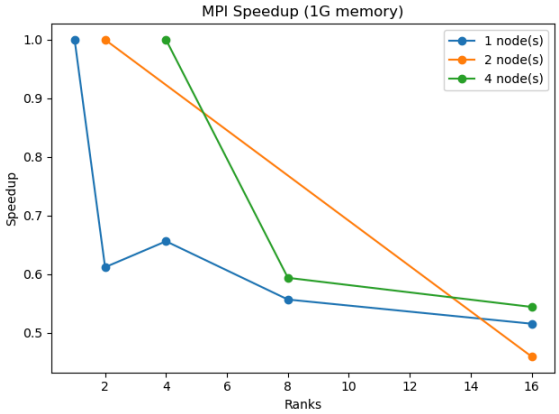
The findings that were most interesting was the time to complete a task seemed independent of input size. As almost any configuration regardless of input size completed quickly ranging from 0.3 seconds to 0.8 seconds (with some spikes going above 1 second).

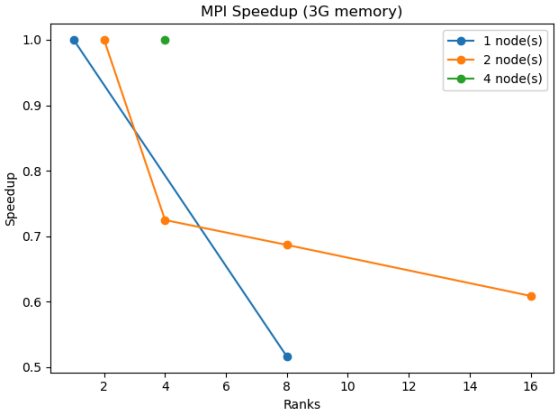
As the graphs will indicate, the increasing of ranks did nothing to improve performance, and in fact seems to slow down in most cases. With the exception of rank 2 which does show improvement some of the time. But this could be showing up in our results due to running on BEOCAT and contention.

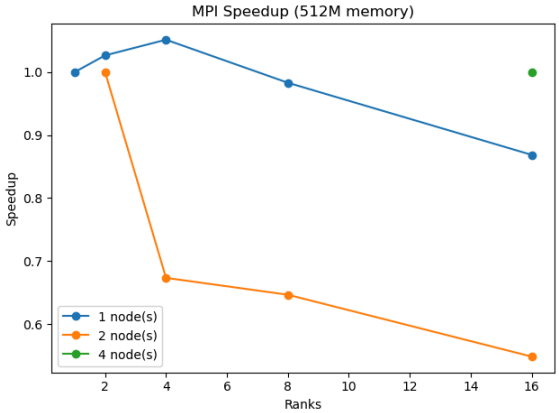
There were some spikes where the time went above 1 second, and these most likely were due to network issues, delays, or contention for nodes.

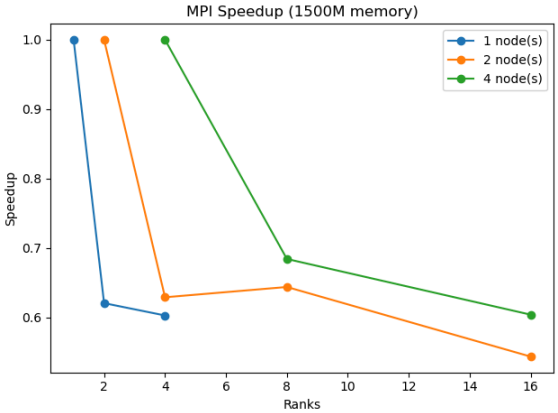
Memory did not seem to impact performance either.

Input size 1000

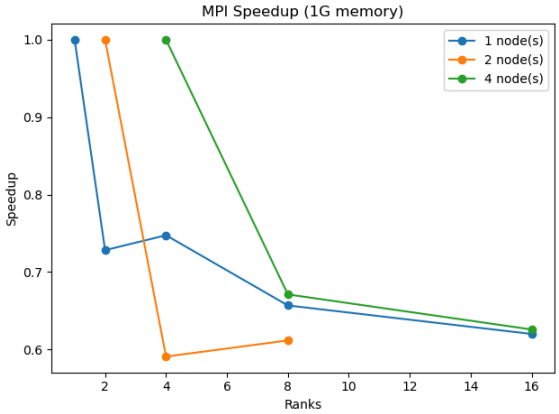


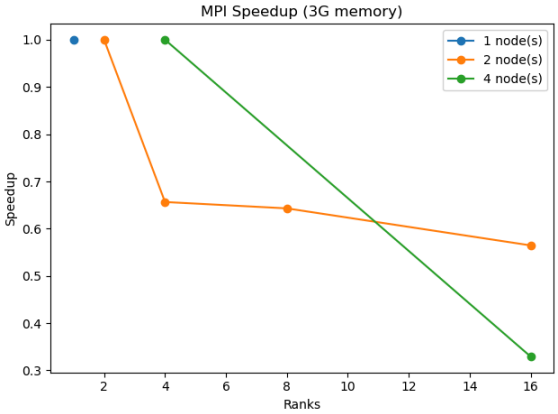


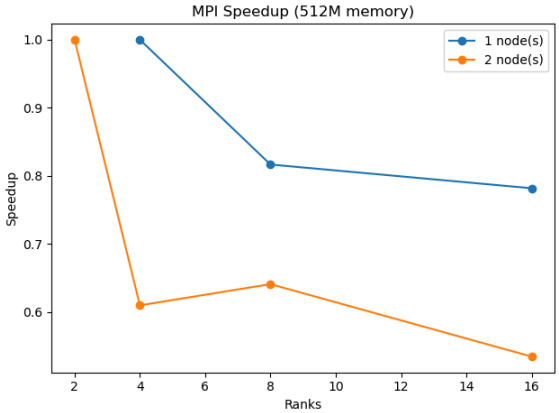


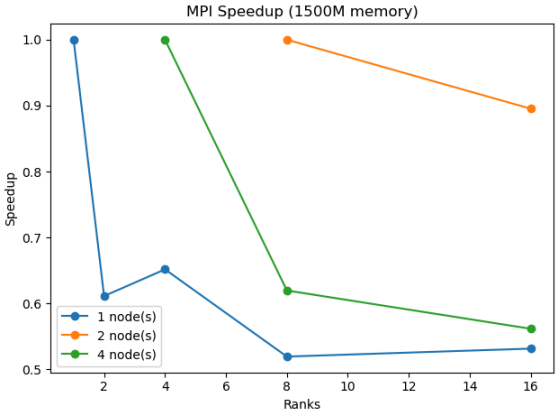


Input size 10,000

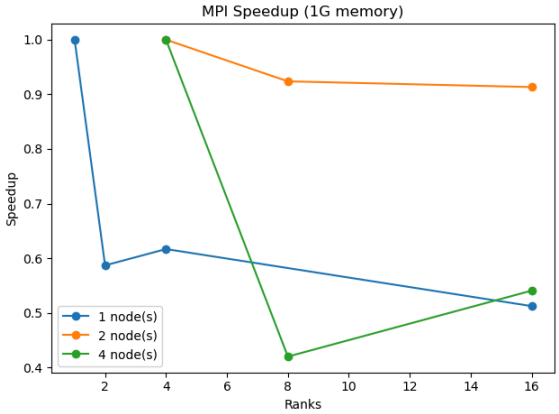


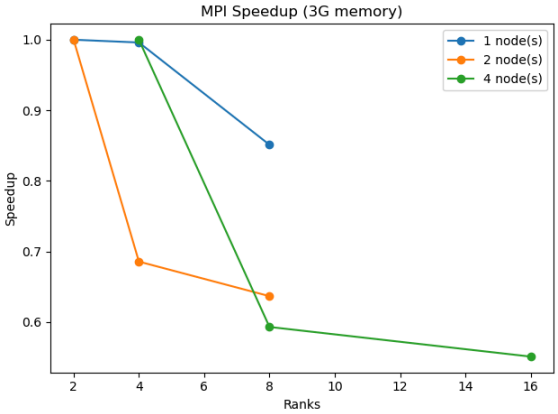


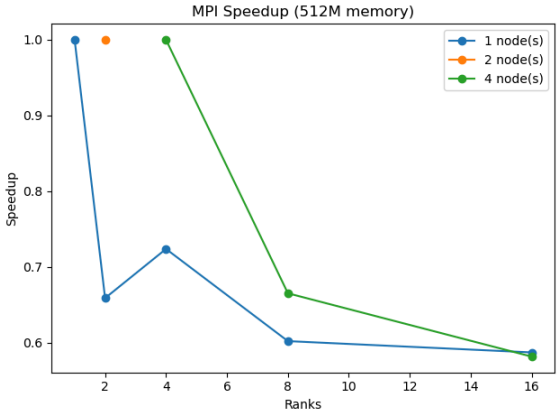


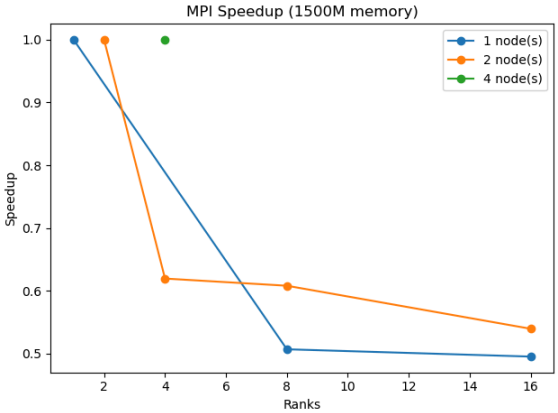


Input size 100,000

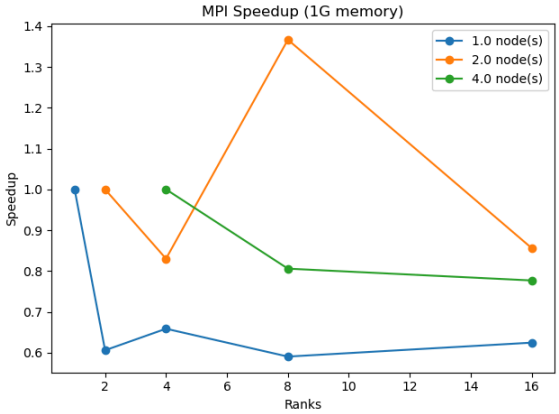


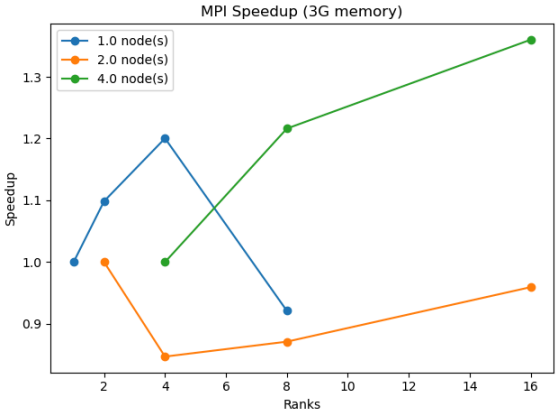


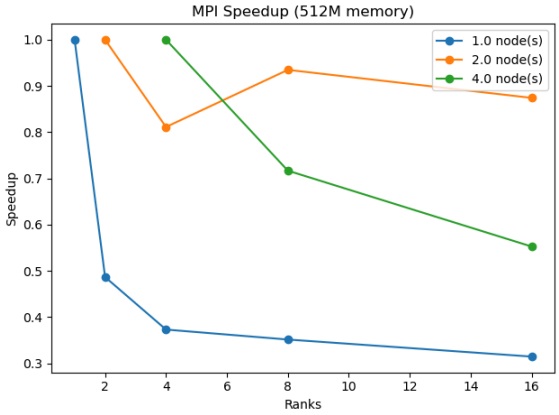


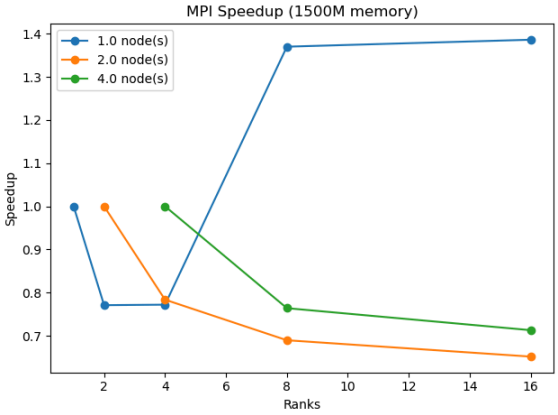


Input size 1,000,000









OpenMP Analysis

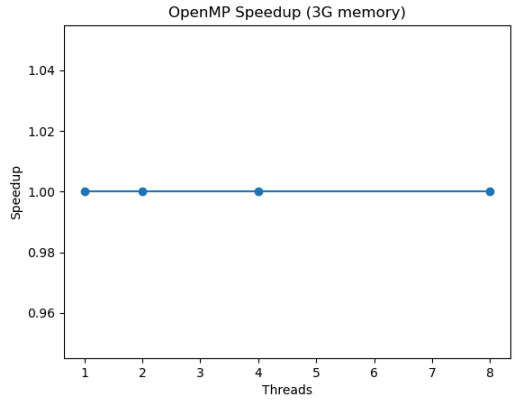
OpenMP runs on one node which differs from the MPI implementation. So the threads are running in parallel on a single node.

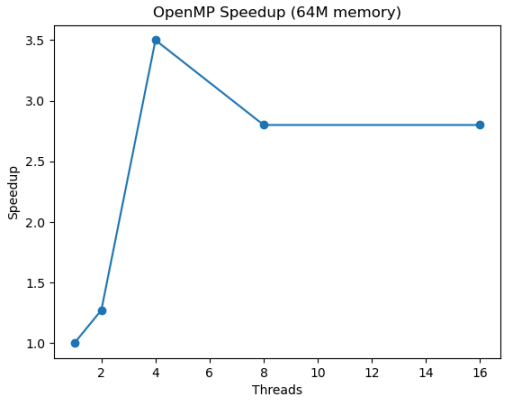
It is significant to note that our previous implementations did not succeed on running 64MB memory allocation where the OpenMP had no issues. This may be due to it reading from one file instead of reading it from multiple nodes.

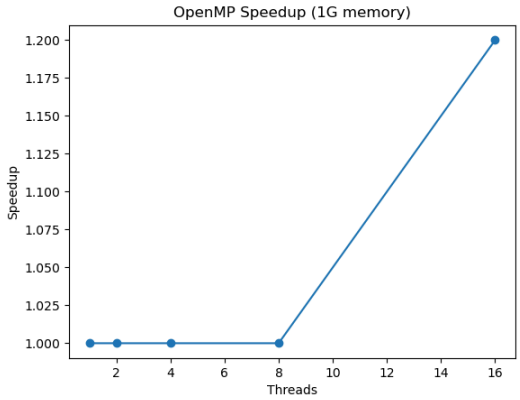
Runtimes were extremely fast on the three smaller input sizes, again compared to MPI in speed. But OpenMP does beat the times by being closer to 0.001 second to 0.01 second. As our graphs below demonstrate we can see clear speedup with the increasing of threads.

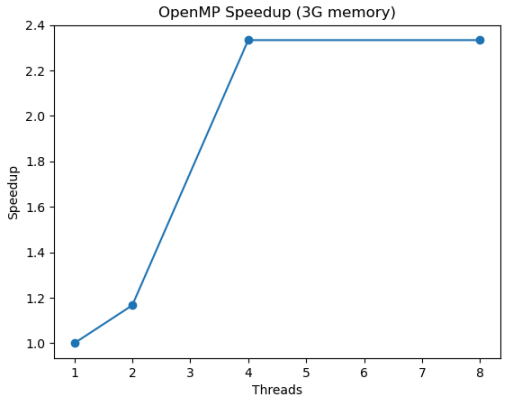
On the largest input size the times were incredibly slow compared to the smaller inputs. Sometimes taking 2 seconds all the way to 109 seconds. 8 Threads seemed to perform the best running around the lower 2 seconds. And the single threaded run took closer to 20 seconds to finish. This is the clearest speedup that we can see on the data that was collected. Especially compared to our previous implementations.

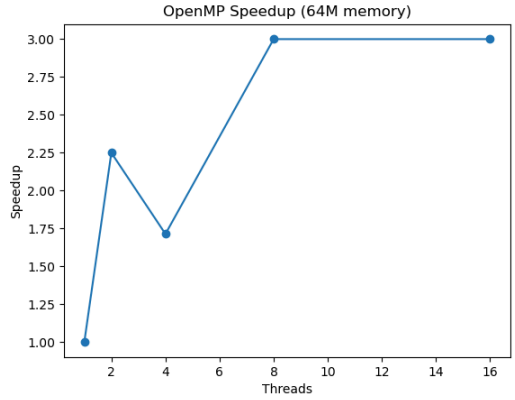
Input size 1,000

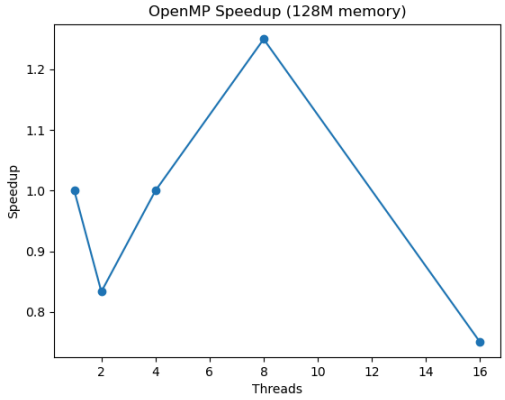




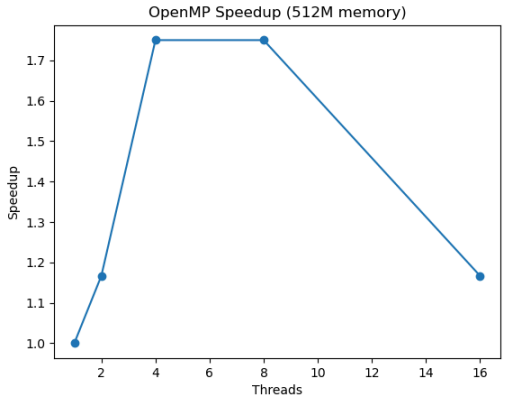


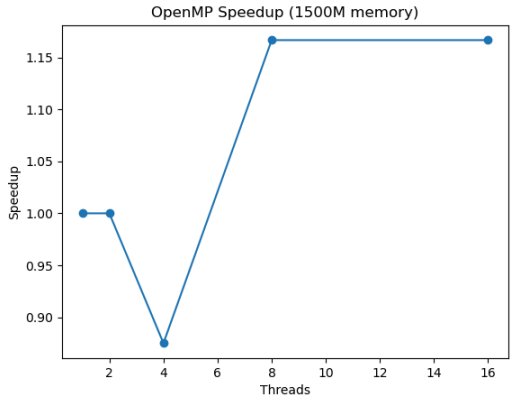


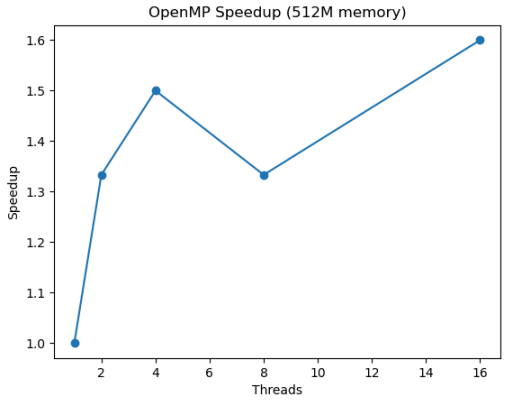


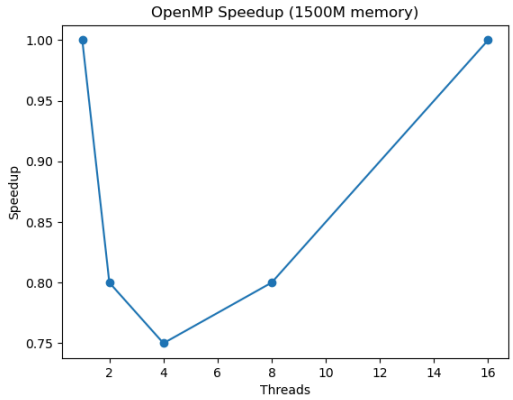


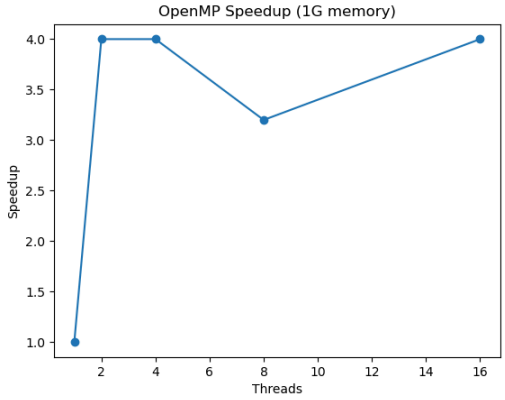
Input size 10,000



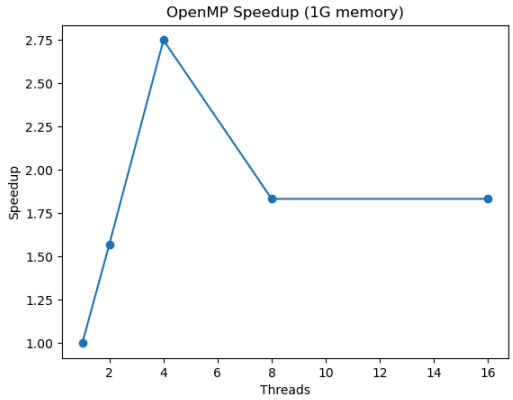


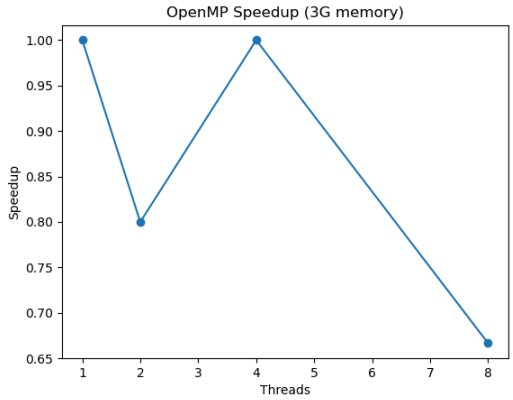


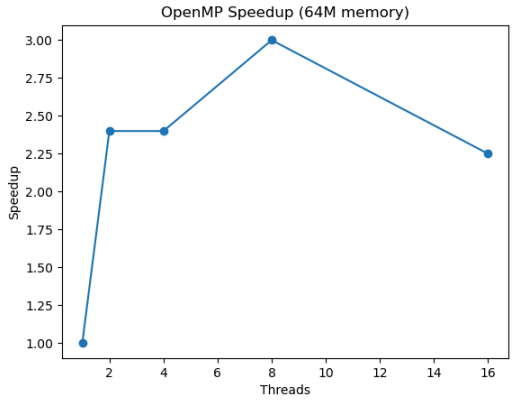


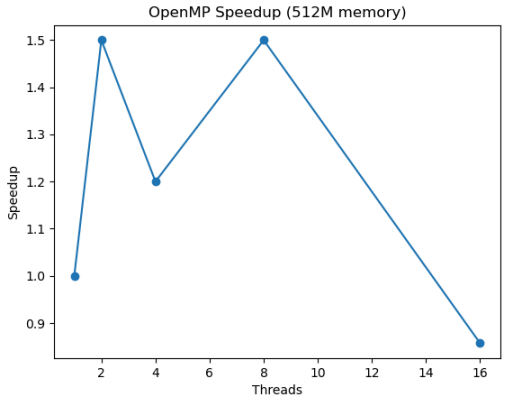


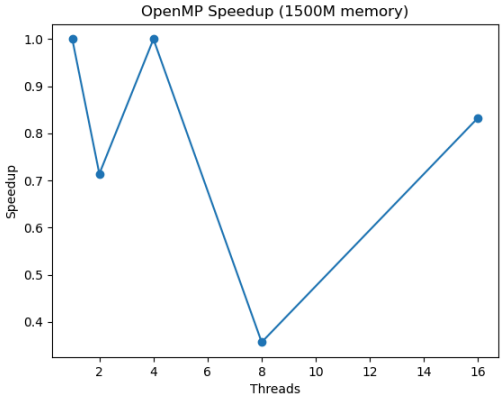
Input size 100,000



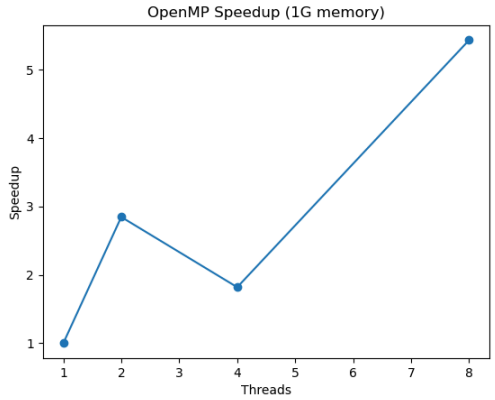


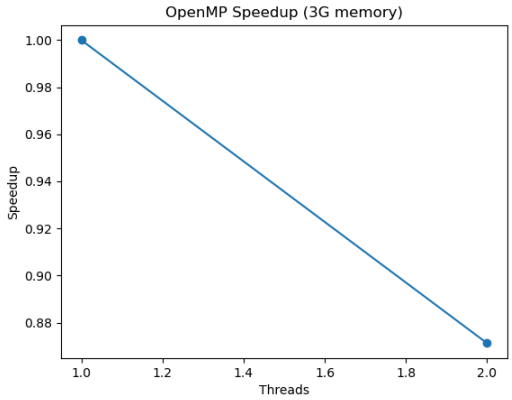


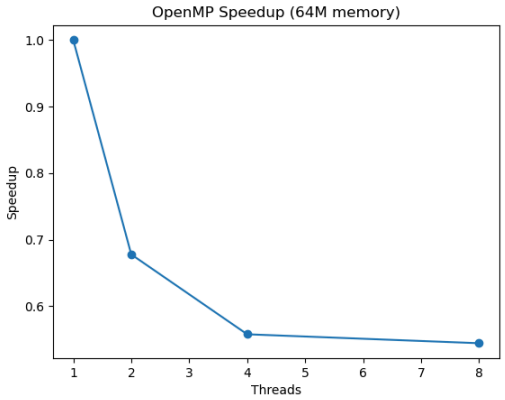


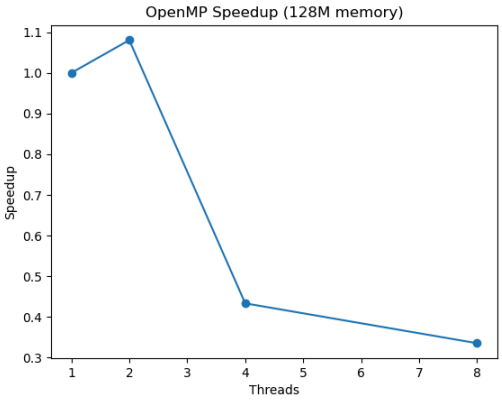


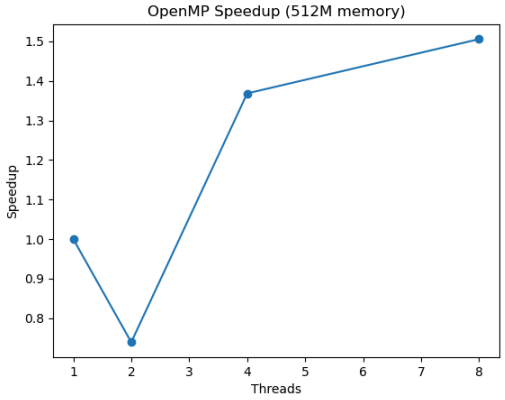
Input size 1,000,000

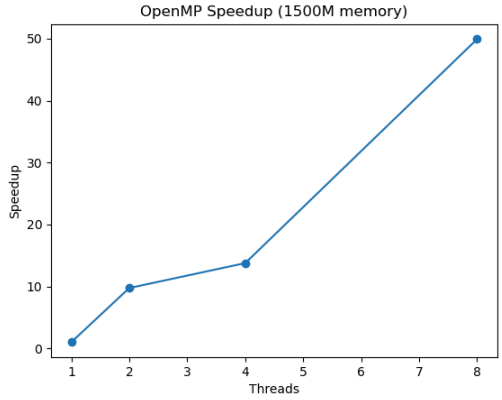












Conclusion

Comparing our combinations of MPI based implementation to our pthreads. There is no significant difference between increasing rank or memory allocation that shows that MPI makes a big difference, whereas our pthreads showed considerable difference between the lower threaded operations.

Comparing the three implementations: pthreads, MPI, and OpenMP. There is a clear winner and loser when looking through our data. Pthreads typically did worse throughout different combinations with some of our best times coming to under two seconds.

References

Kansas State University. (n.d.). Compute Nodes. Beocat. https://support.beocat.ksu.edu/Docs/Compute_Nodes